

~~Task~~
Name: Mohammad Ifaz Alamgir (22241022)

Task 1

First, I created a weighted directed graph. The dequeue function selects the node with the minimum distance from the queue, updating a flag to mark it as visited. The 'dijkstra' function initializes distances to infinity, with the source node's distance set to 0, and it maintains a priority queue of nodes to visit. It iteratively explores neighbours of each node, updating distances if a shorter path is found. The algorithm terminates when all nodes have been visited. The output is written to a file with the shortest distances from the source node to all other nodes. Nodes that are unreachable is marked as -1.

Task 2

Here, the dijkstra function works like before. We will make changes in the dequeue function where we will send a path list and whenever we deque a vertex, we will append

it in the path, that way we will get the shortest path.

We will call the dijkstra function twice with 2 different

Sources the compare the path to find the minimum ~~path~~

time to meet and the vertex where they will meet.

Task 3

This code implements a variation of Dijkstra's Algorithm. Here, instead of iterating until all are visited, it iterates exactly n times, where n is the number of nodes. Within each iteration, it selects the unvisited node with the smallest distance value. If no such node exists, it checks if any nodes are left unvisited, if not, it writes "impossible". It terminates when it reaches the destination node 'h' or all nodes are visited.

During each iteration, it updates the distances and the weight of edge being considered. Finally,

it writes either the shortest distance to the destination

node or Impossible if the destination is unreachable and

the graph contains negative cycles.